WalkGraph MVP Project Brief

1. Project Summary

Project Name: WalkGraph
Working Tagline: A building map you create by walking.

WalkGraph is a node-first indoor navigation and mapping platform that helps users find rooms, offices, halls, staircases, elevators, and accessible routes inside buildings where traditional GPS and Google Maps are not detailed enough.

Unlike conventional indoor mapping systems that require accurate architectural floor plans, WalkGraph allows admins to create a usable indoor map by physically walking through a building and dropping navigation nodes. These nodes represent real-world points such as entrances, corridors, junctions, doors, staircases, elevators, restrooms, offices, and landmarks.

The MVP should focus on creating a functional indoor navigation system using a graph-based model. Outdoor routing can be handled by existing map providers, while indoor routing is powered by WalkGraph's own node and edge system.

The goal is not to build a perfect 3D Google Maps replacement immediately. The goal is to build a practical, human-first indoor navigation tool that works even without detailed floor plans.

---

2. Core Product Philosophy

WalkGraph should prioritize how people actually give and follow directions indoors.

Instead of only saying:

> Walk 24 meters northeast.

The app should be able to say:

> Enter through the main entrance, walk to reception, turn left into the BA corridor, pass BA001 and BA002, then BA003 is the next door on your right.

The system should be based on **walkable routes**, not perfect architectural drawings.

This means the MVP should be:

- Node-first, not floor-plan-first
- Human-readable, not only coordinate-based
- Offline-capable
- Modular and extensible
- Practical for schools, hospitals, churches, campuses, offices, malls, hotels, event centers, and public buildings

3. Problem Statement

Outdoor mapping systems are strong at helping users reach buildings, landmarks, and public places. However, once users enter a building, navigation often breaks down.

Common problems include:

- GPS becomes unreliable indoors
- Large buildings may have many rooms with no public map
- Users do not know which staircase, elevator, corridor, or entrance to use
- Accessibility routes are often unclear
- Visitors may get lost inside schools, hospitals, event centers, office buildings, and campuses
- Building owners may not have professional floor plans available
- Creating detailed indoor maps is usually expensive, slow, or too technical

WalkGraph solves this by allowing buildings to be mapped as connected navigation points.

---

4. Target Users

4.1 Visitors / End Users

These are people trying to find a location inside or around a building.

Examples:

- Students finding lecture rooms
- Patients finding hospital departments
- Church members finding children’s church, media room, or offices
- Event attendees finding halls, booths, restrooms, or exits
- Office visitors finding meeting rooms
- Hotel guests finding facilities

4.2 Building Admins / Mappers

These users create and maintain indoor maps.

Examples:

- School administrators
- Facility managers
- Event organizers
- Church media/admin teams
- Hospital operations staff
- Office managers
- Campus volunteers

4.3 Organization Owners

These users manage buildings, permissions, published maps, and organization-level settings.

Examples:

- University management
- Company admins
- Mall operators
- Hospital management
- J STAR FILMS STUDIOS-style client organizations

---

5. MVP Goal

The MVP should allow an admin to create an indoor map by walking through a building, adding nodes, connecting them, and publishing the map so visitors can search for places and get step-by-step indoor directions.

The MVP should support:

- Outdoor routing to a building
- Indoor node-based mapping
- Multi-floor buildings
- Searchable rooms and landmarks
- Shortest path routing
- Human-readable directions
- Accessibility-aware routing
- Saved/favorite locations
- Offline usage for downloaded buildings
- Basic QR-code-based positioning and recalibration

The MVP should not try to solve full real-time indoor GPS, full 3D scanning, LiDAR mapping, or advanced AR navigation immediately.

---

6. MVP Feature Scope

6.1 Visitor Mode

Visitor Mode is for users who want to find a place.

6.1.1 Find Buildings and Landmarks

Users should be able to:

- Search for a building by name
- Search for nearby supported buildings
- View building details

- Open outdoor route to the building using an external map provider
- See main entrances or recommended entry points

For MVP, outdoor navigation can be delegated to Google Maps, Apple Maps, Mapbox, or OpenStreetMap-based routing.

WalkGraph takes over once the user reaches or enters the building.

6.1.2 Search Indoor Locations

Users should be able to search for:

- Rooms
- Offices
- Lecture halls
- Restrooms
- Staircases
- Elevators
- Entrances
- Exits
- Departments
- Event halls
- Landmarks

Search should support aliases and keywords.

Example:

- “BA001”
- “BA 001”
- “B A one”
- “Computer Lab”
- “Dean’s Office”
- “Media Room”

6.1.3 Choose Starting Point

The app should support multiple starting methods:

1. User manually selects starting node
2. User scans a QR code placed at a known node
3. User selects a nearby entrance
4. User chooses from recently used locations

For MVP, QR/manual start is enough.

6.1.4 Indoor Route Guidance

After choosing a destination and start point, the app should generate a route.

The route should include:

- Current node
- Destination node
- Total estimated distance or step count
- Floor changes
- Stairs/elevator usage
- Human-readable instructions
- Simple visual path between nodes

Example:

> Start at Main Entrance.
> Walk forward to Reception.
> Turn left into BA Corridor.
> Pass BA001 and BA002.
> BA003 is the next door on your right.

6.1.5 Shortest Path Routing

The app should calculate the best path between two nodes.

The default route should prioritize:

- Shortest walkable path
- Clear landmarks
- Lower complexity
- Avoiding restricted areas

Routing should be graph-based.

Nodes represent places. Edges represent walkable paths.

6.1.6 Accessibility-Aware Routing

Users should be able to request accessible routes.

The system should avoid:

- Stairs
- Non-wheelchair-accessible paths
- Restricted corridors
- Narrow/unverified routes if marked

The system should prefer:

- Elevators
- Ramps
- Accessible entrances
- Ground-floor paths

Each node and edge should have accessibility metadata.

6.1.7 Multi-Floor Navigation

The MVP must support buildings with multiple floors.

The app should understand:

- Floor numbers
- Floor names
- Staircase connections
- Elevator connections
- Floor-to-floor transitions

Example:

Staircase A on Floor 0 connects to Staircase A on Floor 1. Elevator B connects Floor 0, Floor 1, Floor 2, and Floor 3.

6.1.8 Save Frequently Visited Places

Users should be able to save places such as:

- Favorite rooms
- Frequent offices
- Lecture halls
- Personal destinations
- Recently visited places

Saved locations should work offline if the building map is downloaded.

6.1.9 Offline Usage

Users should be able to download a building map for offline use.

Offline data should include:

- Building metadata
- Floors
- Nodes
- Edges
- Search index
- Route graph
- QR node references
- Basic visual map assets if available

The app should clearly show whether a building is available offline.

6.2 Mapper/Admin Mode

Mapper Mode is for admins who create and maintain indoor maps.

6.2.1 Create Organization

Admins should be able to create or join an organization.

Organization data should include:

- Organization name
- Logo
- Contact information
- Admin users
- Buildings managed
- Visibility settings

6.2.2 Create Building

Admins should be able to create a building profile.

Building data should include:

- Building name
- Address or description
- GPS location
- Building category
- Number of floors
- Main entrance
- Public/private visibility
- Offline availability setting

6.2.3 Create Floors

Admins should be able to define floors manually.

Each floor should include:

- Floor name
- Floor number
- Optional description
- Optional floor plan image
- Accessibility notes

Examples:

- Ground Floor
- First Floor
- Floor 0
- Floor 1
- Basement
- Mezzanine

6.2.4 Walk-to-Map Node Creation

This is the most important MVP feature.

Admins should be able to walk through a building and add nodes as they move.

Node types should include:

- Entrance
- Exit
- Reception
- Hallway point
- Corridor junction
- Door
- Room
- Office
- Lecture hall
- Restroom
- Staircase
- Elevator
- Ramp
- Landmark
- Restricted area
- Emergency exit

Each node should include:

- Name
- Type
- Floor
- Optional description
- Optional photo
- Optional voice note
- Optional tags
- Accessibility status
- Whether it is searchable
- Whether it can be used as a route checkpoint

6.2.5 Auto-Connect Previous Node

When an admin adds a new node while mapping, the app should ask whether to connect it to the previous node.

Example flow:

1. Add Main Entrance
2. Walk to Reception
3. Add Reception
4. App asks: “Connect Main Entrance to Reception?”
5. Admin confirms

This creates an edge.

6.2.6 Manual Node Connection

Admins should also be able to manually connect nodes.

This is necessary for:

- Corridors with multiple doors
- Loops
- Alternate routes
- Elevators
- Staircases
- Emergency exits
- Doors that connect two halls

6.2.7 Edge Metadata

Each connection between nodes should store route information.

Edge metadata should include:

- Distance estimate
- Walk time estimate
- Direction hint
- Accessibility status
- Stairs involved
- Elevator involved
- Ramp involved
- Restricted access
- One-way status if relevant
- Indoor/outdoor transition

For MVP, distance can be approximate.

6.2.8 Staircase and Elevator Mapping

Admins should be able to map vertical movement.

A staircase or elevator should be represented as connected nodes across floors.

Example:

- Staircase A - Ground Floor
- Staircase A - First Floor
- Staircase A - Second Floor

These are separate nodes connected by vertical edges.

Vertical edges should indicate:

- Stairs/elevator/ramp
- Accessibility
- Estimated time
- Connected floors

6.2.9 QR Code Generation

The system should generate QR codes for important nodes.

QR codes should be printable and placeable at:

- Main entrances
- Floor landings
- Reception
- Staircases
- Elevators
- Major corridor junctions
- Large halls

When scanned, the QR code should tell the app exactly where the user is.

QR codes should encode or reference:

- Organization ID
- Building ID
- Floor ID
- Node ID

6.2.10 Route Testing Mode

Admins should be able to test a route before publishing.

Example:

- Select start node
- Select destination node
- Generate route
- Walk the route
- Confirm whether instructions make sense
- Edit unclear nodes or connections

This is important because node-based maps rely heavily on good human-readable labels.

6.2.11 Publish / Unpublish Building Maps

Admins should be able to control whether a building map is:

- Draft
- Private
- Organization-only
- Public
- Archived

For MVP, use simple states:

- Draft
- Published

- Archived

---

7. Optional MVP Enhancements

These are useful, but not strictly required for the first build.

7.1 Optional Floor Plan Overlay

Admins may upload a floor plan image or simple sketch.

The node graph can be placed over it, but the system should not require it.

7.2 Basic Compass/Heading Guidance

The visitor app may use the phone compass to show approximate orientation.

Example:

- “Face toward Reception”
- “Turn left”
- “Continue forward”

This should be treated as helpful but not guaranteed.

7.3 Step-Based Progress Estimation

The app may use basic phone motion sensors to estimate progress between nodes.

This can support simple messages like:

- “You should be near BA002 now”
- “Next checkpoint: Staircase A”

This should not be treated as perfect indoor GPS.

7.4 Photos for Landmarks

Admins may attach photos to nodes.

Example:

- Photo of reception desk
- Photo of staircase entrance
- Photo of room door

This helps visitors confirm they are in the right place.

---

8. Explicitly Out of Scope for MVP

The following should not be part of the MVP unless there is extra time and budget:

- Full 3D building scanning
- LiDAR-based mapping
- Full AR indoor navigation
- Perfect real-time indoor GPS
- Automatic room detection from camera
- Complex crowd-aware routing
- Emergency evacuation simulation
- Advanced computer vision landmark recognition
- Wi-Fi fingerprint positioning
- Bluetooth beacon deployment system
- Public marketplace for building maps
- Full OpenStreetMap import/export system

These can become later-stage features.

---

9. Suggested Product Modules

The system should be modular from day one.

9.1 Auth Module

Handles:

- User login
- Admin login
- Organization roles
- Permissions

9.2 Organization Module

Handles:

- Organizations
- Buildings
- Team members
- Visibility rules

9.3 Building Module

Handles:

- Building profiles
- Floors
- Entrances
- Metadata

9.4 Node Mapping Module

Handles:

- Node creation
- Node editing
- Node types
- Node metadata
- Photos/attachments

9.5 Routing Graph Module

Handles:

- Edges
- Pathfinding
- Accessibility routing
- Multi-floor routing
- Route validation

9.6 Search Module

Handles:

- Searchable rooms
- Keywords
- Aliases
- Recent searches
- Saved locations

9.7 Offline Module

Handles:

- Downloaded building maps
- Offline search
- Offline routing
- Local cache sync

9.8 QR Positioning Module

Handles:

- QR code generation
- QR scanning
- Node-based recalibration
- Printed checkpoint support

9.9 Admin Review Module

Handles:

- Draft maps
- Route testing
- Publishing
- Versioning

---

10. Suggested Tech Stack

10.1 Recommended MVP Stack

Frontend Web Admin

Recommended:

- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui
- React Hook Form
- Zod

Why:

- Fast to build
- Strong developer ecosystem
- Good for dashboards and admin tools
- Easy deployment on Vercel
- Works well with TypeScript-heavy architecture

Mobile App

Recommended:

- React Native with Expo
- TypeScript
- Expo Router
- SQLite for local/offline data
- Expo Camera or compatible QR scanning library
- Device motion/compass APIs for later sensor support

Why:

- Faster MVP development
- One codebase for Android and iOS
- Good enough access to camera, location, storage, and sensors
- Easier iteration than native development

Alternative:

- Flutter

Flutter is also a strong option, especially for polished mobile UI, but if the team is already stronger in TypeScript/React, React Native is the better MVP choice.

Backend

Recommended:

- Node.js / TypeScript
- Next.js API routes or separate NestJS/Fastify backend
- Prisma ORM
- PostgreSQL
- PostGIS extension if geospatial queries become important

For MVP, plain PostgreSQL is enough. PostGIS becomes more useful once outdoor/geo queries, spatial search, and map overlays become deeper.

Database

Recommended:

- PostgreSQL for production
- Neon, Supabase Postgres, Railway Postgres, or managed Postgres provider
- SQLite on mobile for offline data

Core entities:

- Users
- Organizations
- Buildings
- Floors
- Nodes
- Edges
- Routes
- Saved places
- QR checkpoints
- Map versions

Maps

Recommended:

- MapLibre GL for open map rendering
- OpenStreetMap tiles or a commercial tile provider
- Optional Mapbox if budget and design polish matter
- External deep links to Google Maps/Apple Maps for outdoor navigation

For MVP, do not overbuild outdoor routing. Use deep links.

Routing Engine

Recommended:

- Custom graph routing in TypeScript
- Dijkstra for MVP
- A* later if coordinates become stronger

The routing graph should be separate from the visual map.

File Storage

Recommended:

- Cloudflare R2
- S3-compatible storage
- UploadThing or similar upload helper for quick MVP

Used for:

- Node photos
- Building images
- Optional floor plan images
- Organization logos

Authentication

Recommended:

- Clerk
- Auth.js
- Supabase Auth
- Firebase Auth

If speed matters, Clerk is probably the smoothest. If cost/control matters, Auth.js with Postgres is fine.

Deployment

Recommended:

- Vercel for web/admin/backend if using Next.js
- Expo Application Services for mobile builds
- Neon or managed Postgres for DB
- Cloudflare R2 for file storage

---

11. Data Model Overview

11.1 Organization

Represents a company, school, church, hospital, or institution.

Fields:

- id
- name
- slug
- logoUrl
- ownerId
- visibility
- createdAt
- updatedAt

11.2 User

Fields:

- id
- name
- email
- role
- createdAt
- updatedAt

11.3 OrganizationMember

Fields:

- id
- organizationId
- userId
- role
- permissions

Roles:

- Owner
- Admin
- Mapper
- Viewer

11.4 Building

Fields:

- id
- organizationId
- name
- slug
- description
- address
- latitude
- longitude
- status

- visibility
- mainEntranceNodeId
- createdAt
- updatedAt

11.5 Floor

Fields:

- id
- buildingId
- name
- levelNumber
- description
- floorPlanImageUrl
- createdAt
- updatedAt

11.6 Node

Fields:

- id
- buildingId
- floorId
- name
- type
- description
- x
- y
- latitude
- longitude
- heading
- photoUrl
- searchable
- tags
- aliases
- accessibilityFlags
- restricted
- createdAt
- updatedAt

Note:

For MVP, x/y can be optional or relative. A pure graph can work even without coordinates, but approximate coordinates help visual layout later.

11.7 Edge

Fields:

- id

- buildingId
- fromNodeId
- toNodeId
- distanceEstimate
- walkTimeEstimate
- directionHint
- accessible
- requiresStairs
- requiresElevator
- restricted
- oneWay
- createdAt
- updatedAt

11.8 QRCheckpoint

Fields:

- id
- buildingId
- floorId
- nodeId
- code
- label
- active
- createdAt
- updatedAt

11.9 SavedPlace

Fields:

- id
- userId
- buildingId
- nodeId
- label
- createdAt

11.10 MapVersion

Fields:

- id
- buildingId
- versionNumber
- status
- createdBy
- publishedAt
- createdAt

---

12. Routing Logic

The MVP routing engine should operate on graph data.

12.1 Nodes

Nodes represent important indoor positions.

Examples:

- Main Entrance
- Reception
- Corridor Junction A
- BA001
- BA002
- Staircase A Floor 0
- Staircase A Floor 1
- Elevator B Floor 2

12.2 Edges

Edges represent valid movement between nodes.

Examples:

- Main Entrance → Reception
- Reception → Corridor Junction A
- Corridor Junction A → BA001
- BA001 → BA002
- Staircase A Floor 0 → Staircase A Floor 1

12.3 Pathfinding

Use Dijkstra for MVP.

Each edge should have a weight.

Default weight can be based on:

- Distance estimate
- Walk time estimate
- Complexity penalty
- Accessibility penalty
- Stair penalty
- Restricted path penalty

For accessible routing, the engine should exclude or penalize non-accessible edges.

12.4 Instruction Generation

After route calculation, the app should convert node paths into readable instructions.

Example route:

Main Entrance → Reception → BA Corridor → BA001 → BA002 → BA003

Generated instructions:

1. Start at Main Entrance.
2. Walk forward to Reception.
3. Turn left into BA Corridor.
4. Pass BA001 and BA002.
5. BA003 is the next door on your right.

Instruction quality depends heavily on good node naming and edge direction hints.

Admins should be allowed to manually edit direction hints.

---

13. Offline Strategy

Offline support should be included early because indoor navigation may happen in buildings with poor network coverage.

13.1 Downloadable Building Package

A building package should include:

- Building metadata
- Floors
- Nodes
- Edges
- QR checkpoints
- Search aliases
- Route graph
- Optional images
- Optional floor plan

13.2 Local Storage

On mobile, store offline packages in SQLite.

13.3 Sync Strategy

For MVP:

- User manually downloads a building
- App checks for updated versions when online
- User can update downloaded map
- If offline, app uses the last downloaded version

Avoid complex real-time sync for MVP.

---

14. Permissions and Roles

14.1 Owner

Can:

- Manage organization
- Add/remove admins
- Create buildings
- Publish maps
- Archive maps

14.2 Admin

Can:

- Create/edit buildings
- Create/edit floors
- Create/edit nodes
- Create/edit routes
- Generate QR codes
- Publish drafts if permitted

14.3 Mapper

Can:

- Add/edit nodes
- Add/edit edges
- Upload photos
- Test routes

14.4 Visitor

Can:

- Search public buildings
- Download maps
- Navigate routes
- Save places
- Scan QR codes

---

15. Suggested MVP Screens

15.1 Mobile Visitor Screens

1. Home / Search
2. Nearby Buildings
3. Building Details

4. Indoor Search
5. Select Starting Point
6. Scan QR Code
7. Route Preview
8. Step-by-Step Navigation
9. Saved Places
10. Offline Downloads

15.2 Mobile Mapper Screens

1. Mapper Dashboard
2. Select Organization
3. Select Building
4. Select Floor
5. Start Mapping Session
6. Add Node
7. Connect Node
8. Add Edge Details
9. Test Route
10. Generate QR Code

15.3 Web Admin Screens

1. Organization Dashboard
2. Buildings List
3. Building Detail
4. Floors Manager
5. Nodes List
6. Graph/Map Editor
7. Route Tester
8. QR Code Manager
9. Publish Settings
10. Team/Roles

---

16. Build Phases

Phase 1: Core Data and Admin Foundation

Deliverables:

- Authentication
- Organization creation
- Building creation
- Floor creation
- Node creation
- Edge creation
- Basic web admin dashboard

Goal:

Admins can manually create a building graph.

Phase 2: Routing Engine

Deliverables:

- Dijkstra routing
- Route preview
- Human-readable instructions
- Multi-floor routing
- Accessibility-aware route filtering

Goal:

Users can get routes between indoor locations.

Phase 3: Mobile Visitor App

Deliverables:

- Building search
- Indoor location search
- Start/destination selection
- Route display
- Saved places
- Basic offline downloads

Goal:

Visitors can navigate buildings.

Phase 4: Mobile Mapper Mode

Deliverables:

- Walk-to-map flow
- Add node while walking
- Connect previous node
- Add photos
- Add accessibility tags
- Test route

Goal:

Admins can create maps from inside the building.

Phase 5: QR Positioning

Deliverables:

- Generate QR codes for nodes
- Scan QR code to set current location
- Use QR scan for route recalibration
- Printable QR sheet

Goal:

Users can start navigation accurately from known checkpoints.

Phase 6: Polish and Offline Reliability

Deliverables:

- Offline building package system
- Local SQLite cache
- Update detection
- Error handling
- Better search aliases
- Route instruction improvements

Goal:

The MVP becomes reliable enough for pilot testing.

---

17. MVP Success Criteria

The MVP is successful if:

1. An admin can create a building with multiple floors.
2. An admin can walk through the building and add connected nodes.
3. An admin can mark rooms, entrances, stairs, elevators, and accessible paths.
4. A visitor can search for a room or location.
5. A visitor can choose or scan their starting location.
6. The app can generate a valid indoor route.
7. The route can include floor changes.
8. The route can avoid stairs when accessibility mode is enabled.
9. The visitor can save frequently visited places.
10. The visitor can download a building map and use it offline.

---

18. Main Risks

18.1 Indoor Positioning Accuracy

Phone sensors alone are not reliable enough for perfect indoor GPS.

Mitigation:

Use QR checkpoints and manual start points for MVP.

18.2 Bad Map Quality

If admins create poor nodes or bad labels, route instructions become confusing.

Mitigation:

Add route testing mode and clear mapping guidelines.

18.3 Overbuilding Too Early

3D scanning, AR, and real-time positioning can delay the MVP.

Mitigation:

Focus on graph-based routing first.

18.4 Offline Complexity

Sync conflicts can become complicated.

Mitigation:

Use versioned building packages and manual update prompts for MVP.

18.5 Accessibility Responsibility

Incorrect accessibility data can harm user trust.

Mitigation:

Allow admins to mark routes as verified or unverified.

---

19. Future Features

After MVP, consider adding:

- AR arrows
- Bluetooth beacon positioning

- Wi-Fi fingerprinting
- Visual landmark recognition
- Floor plan auto-tracing
- OpenStreetMap indoor export/import
- Public map marketplace
- Analytics for most searched rooms
- Emergency evacuation routes
- Event-specific temporary maps
- Voice-guided navigation
- Crowd-aware routing
- Multi-language directions
- Admin approval workflows
- Map contribution system
- API for third-party apps

---

20. Recommended MVP Positioning

WalkGraph should not be positioned as a replacement for Google Maps.

It should be positioned as:

> A simple indoor navigation system for buildings Google Maps does not understand.

Or:

> A map you create by walking through your building.

Or:

> Indoor directions without needing a professional floor plan.

The strongest early markets are:

- Universities
- Churches
- Hospitals
- Event centers
- Office complexes
- Large schools
- Conference venues

---

21. Final MVP Definition

The first version of WalkGraph should be a practical indoor navigation tool where:

- Admins create buildings and floors
- Admins walk through spaces and add nodes

- Nodes represent real-world places
- Edges represent walkable paths
- QR codes anchor accurate starting points
- Users search destinations
- The app generates shortest paths
- Directions are human-readable
- Accessibility routes are supported
- Building maps can be used offline

This is the smallest version that proves the product idea without getting trapped in impossible indoor GPS, expensive 3D scanning, or overcomplicated map rendering.

The MVP should feel like:

> Google Maps gets you to the building. WalkGraph gets you to the room.